1 Overview

The design of the runtime will be based on the 1992 reference [1] plus the augmentation to use Apply/Eval approach from the 2006 reference [2]. We will also use a single unified stack.

2 Frame Types

Using one unified stack by marking boundaries of parts of the stack by frames means no standalone items (such as arguments) will exist on its own in the stack; instead they will be described by frame headers. When accessing the stack, we will always be able to inspect a frame header that is present on top of the stack before manipulating the stack. The variants of frame are:

- Continuation Frame
- Application Frame
- Update Frame

For each of the above types of frames, a stack frame header is embedded (pointer to some meta-info describing items associated with the frame that are also pushed earlier onto the stack). The stack frame meta-info is used by the garbage collector (GC) to discriminate pointer/non-pointer types (GC will use this to trace live objects instead of using a separate stack as roots of live objects as in the approach of the 1992 reference [1]).

These frames are produced and consumed during certain state transitions that are mentioned in the following sections of this document.

3 Closure Layout in Memory

generic and uniform layout for all closures:

- info pointer
- continugous section of pointers
- continugous section of non-pointers

where info pointer points to a data structure containing:

- entry code
- GC codes
- possibly other meta-info

4 Continuation Frame

4.1 triggered by

start of evaluation of a case expression

4.2 actions to the stack

push each local env variable that are live across the continuation

push a Continuation Frame that is embedded with:

- stack frame info containing previously inserted items onto the stack
- return address/vector corresponding to the alternatives

4.3 consumed by

an eventual return of constructor transfers control to the continuation (for example, one can possibly use the convention of return vector jump indexed by the constructor variant), where it pops from the stack:

- continuation frame
- local vars associated with the continuation frame (these are read and used for the continuation)

before proceeding to execute the remainder of the continuation

5 Application Frame

5.1 triggered by

a function application, and before evaluation of the function to weak head normal form

5.2 actions to the stack

push each argument into the stack

push an Application Frame that also embedded with:

- stack frame info for previously inserted items onto the stack

5.3 consumed by

before an enter of a closure/function, an arity check is run using the application frame info that already encodes available arguments on the stack

different scenarios to consider:

1) if the closure finds over-saturation of arguments:

copy and remove the function's required arguments off the stack and adjust the application frame to encode the remaining arguments on the stack, before jumping to the function's entry code with the arguments (eg: a possible calling convention could be leaving those arguments on the stack)

2) if the closure finds exact number of arguments avaialable from the application frame:

pop the application frame off the stack, copy and pop all the arguments (associated with the application frame) off the stack, then jump to the function's entry code with the arguments

3) if the closure finds under-saturation of arguments:

create a new partial application (PAP) closure on the heap taking the existing function closure and available arguments from the stack as payload on the heap, pop the application frame off the stack and all the arguments off the stack, and return the new PAP closure (for example, by saving its address in a designated register)

6 Update Frame

6.1 triggered by

when thunk / updatable closure is entered for the first time, where entry code of the thunk does the push of an Update Frame onto the stack

note: a thunk is created by at a let/letrec binding (includes saving the free variables into the payload of the thunk on the heap)

6.2 actions to the stack

push an Update Frame that also embedded with:

- stack frame info
- address of the thunk / updatable closure in the heap

6.3 consumed by

either:

- a return of constructor, or
- a completion of a partial application (where a previous removal of an application frame from the stack happened)

sees the update frame on top of the stack

we read the address of the thunk associated with the Update Frame and update the thunk with a target closure by overwriting the thunk with an indirection closure

notes about indirection closure:

it is a closure with:

- 1 payload field that is the address of the target closure
- entry code that jumps to the target closure (its entry code)

this means a thunk / any closure needs to be atleast 2 words in size (1 payload field + a pointer to its entry code)

the indirection closure can be removed during a GC sweep since we can use the target closure pointer and remove the intermediate indirection

7 States

a tuple of:

- code
- $s \equiv$ unified stack: containing a frame of type: continuation/update/application plus any relevant payloads associated with the frame
- $h \equiv$ heap: for locals, closures
- $\sigma \equiv$ globals: for top-level variables/binders

8 State Transition: eval case expression

we enqueue live local variables into the stack along with a continuation frame

```
eval (case e of alts) ρ, s, h, σ

⟹

eval e ρ, s', h, σ

where:

s' = (cf:vs) ++ s

vs = local live vars for alternatives (
    equivalent to providing ρ for continuation)

cf = continuation frame (layout-aware of its 'vs
    ' payload)
```

9 State Transition: eval let/letrec expression

this is largely same as original transition in reference [1]

notes:

free variables are stored into the payload along with each closure on the heap

thunks can be created from let bindings

for letrec:

```
eval (letrec x1 = vs1 \π1 xs1 ↦ e1; ...; in e)
    ρ,
s,
h,
σ

⟹

eval e ρ', s, h', σ

where:

  ρ' = ρ[x1 ↦ addr a1, ...]

  h' = h[a1 ↦ (vs1 \π1 xs1 ↦ e1)
         (ρrhs vs1);
         ...]

  ρrhs = ρ'
```

10 State Transition: eval constructor

largely the same transition as descrbied in the 1992 reference [1]

```
eval (c xs) ρ, s, h, σ

⟹

ReturnCon c xs (val ρ σ xs), s, h, σ
```

11 State Transition: ReturnCon with Continuation Frame on top of the stack

```
ReturnCon c ws, ((cf:locals)++s), h, σ

where:

  cf = continuation frame

⟹

eval e ρ', s, h, σ

where:
  e = expression to eval after jump to
      continuation (eg: jump to fr.return_vector
      [offset])

  offset = computable based on constructor
      variant 'c' (or tag depending on
      convention)

  ρ' = ρ ∪ {vs ↦ ws}

  ρ = popped 'locals' from continuation frame
```

for default pattern match: don't need to add additional {vs ↦ ws} binding to local env.

for match-all variable binder pattern: create a closure in the augmented heap h' for the constructor 'c', introduce a binding to the created closure in the local env., use 'ws' as payload to the closure in order to bind to the constructor parameters

12 State Transition: eval literal

similar logic as constructor

```
eval k ρ, s, h, σ

⟹

ReturnInt k, s, h, σ
```

eval of a variable bound to a primitive value:

```
eval (f {}) ρ[f ↦ Int k], s, h, σ

⟹

ReturnInt k, s, h, σ
```

13 State Transition: ReturnInt with Continuation Frame on top of the stack

```
ReturnInt k, (cf:locals)++s, h, σ

where:

  cf = continuation frame

⟹

eval e ρ, s, h, σ

where:

  e = expression to eval after jump to
      continuation

  ρ = popped 'locals' from continuation frame
```

handling alternative with match-all variable binder x:

create additional local variable that binds to Int primitive:

```
ρ = popped 'locals' from continuation frame
        ∪ {x ↦ Int k}
```

14 State Transition: eval builtin operator

for example, binary operator ⊕ with primitive int parameters:

```
eval ⊕{x1, x2}
  ρ[x1 ↦ Int i1, x2 ↦ Int i2],
s,
h,
σ

⟹

ReturnInt (⊕ i1 i2), s, h, σ
```

15 State Transition: ReturnPAP with Continuation Frame on top of the stack

this is disallowed since case scrutinee needs to be a ADT or primitive literal

16 State Transition: eval function application

```
eval (f xs) ρ, s, h, σ

⟹

// construct the application frame before eval
    of f to weak head normal form
eval f ρ, (af:xs)++s, h, σ

where:

  af ≡ application frame

  xs ≡ arguments associated with function
       application
```

after evaluation of a value to weak head normal form, if that value is determined to be a function-like value:

```
// inspect meta-info of the object to detect
    that its type is a function-like value
ReturnFun a / ReturnPAP a, (af:xs)++s, h, σ

where:

  a ≡ address of functional type that has
      already been evaluated to weak head normal
       form

  af ≡ application frame

  xs ≡ arguments associated with the application
       frame

⟹

// exact arguments saturation
if arity(f) = len(xs):

enter a with xs, s, h, σ

---

// over-saturation of arguments
if arity(f) < len(xs):

enter a with xs1, (af':xs2)++s, h, σ

  where:

    xs1++xs2 = xs, len(xs1) = arity(f)

    af' ≡ adjusted application frame describing
         over-saturated/remaining arguments 'xs2'

---
// under-saturation of arguments
if arity(f) > len(xs):

// pop application frame off the stack
//
// PAP closure with address a'
//
// give control to whatever is next on the stack
     (eg: continuation / update)
ReturnPAP a', s, h', σ

  where:

    // create a PAP closure with address a' in
        the heap and take available args and
        function from the original stack into
        closure's payload
    h' ≡ h[a' ↦ (f':vs1 \n vs2 ↦ f' (vs1++vs2)
        ) a:xs]

    // 'xs' are originally arguments on the
        stack but transferred in as free
        variables in the closure's payload
    len(xs) = len(vs1)

    // 'vs2' are the remaining arguments for
        function f
    len(vs1) + len(vs2) = arity(f)
```

17 State Transition: enter non-updatable closure

```
// alternatively put args on stacks for calling
    convention
enter a with args,
s,
h[a ↦ (vs \n xs ↦ e) ws_f],
σ

⟹

eval e ρ, s, h, σ

where:

  ρ = {vs ↦ ws_f, xs ↦ args}
```

18 State Transition: enter indirection closure

```
enter a,
s,
h[a ↦ indirection(a_target)],
σ

⟹

enter a_target, s, h, σ
```

19 State Transition: enter an updatable closure

execute thunk's entry code, which does:

pushes an Update Frame onto the stack

overwrite its own info table to point to a blackhole (indicating that the thunk has entered but has not evaluated to completion; possible use this to detect cycle and/or for queuing for multi-threaded runtime)

```
enter a,
s,
h[a ↦ (vs \u {} ↦ e_thunk) free_vars],
σ

where:

  a is a thunk

⟹

// thunk's entry code pushes update frame
    embedded with thunk a onto the stack, and
    places a blackhole, and continues to execute
     thunk's body e_thunk
eval e_thunk ρ,
uf:s,
h',
σ

where:

  uf ≡ update frame associated with the thunk a

  ρ[vs ↦ free_vars] ≡ captured free_vars needed
       by the thunk

  // temporarily update thunk's info pointer to
      blackhole, but keep payload
  h' = h[a_thunk ↦ blackhole_with_payload(
      free_vars)]
```

20 State Transition: ReturnCon with Update Frame on top of the stack

the thunk body eventually evaluates to weak head normal form (in this case a constructor) where the control lands on the Update Frame sitting on top of the stack

the Update Frame's thunk is overwritten with an indirection pointer to point to the evaluated value, and the Update Frame is popped from the stack

the evaluated value is further returned to the outer context

```
ReturnCon c ws, (uf:s), h, σ

⟹

ReturnCon c ws, s, h'', σ

where:

  uf ≡ update frame

  a_thunk ≡ uf.thunk

  // introduce a closure for the constructor
  h' = h[a_c ↦ ({xs} \n {} ↦ c xs) ws]

  // overwrite thunk to point to the evaluated
      value
  h'' = h'[a_thunk ↦ indirection_closure(a_c)]
```

21 State Transition: ReturnPAP with Update Frame on top of the stack

the thunk body eventually evaluates to a weak head normal form (in this case a partial application) and the control lands on the Update Frame sitting on top of the stack

the Update Frame's thunk is overwritten with an indirection pointer to point to the evaluated value, and the Update Frame is popped from the stack

the evaluated value is further returned to the outer context

```
ReturnPAP a_pap, (uf:s), h, σ

⟹

// pop update frame
//
// return evaluated PAP to outer context
ReturnPAP a_pap, s, h', σ

where:

  uf ≡ update frame

  a_thunk ≡ uf.thunk

  // overwrite thunk to point to the evaluated
      value
  h' = h[a_thunk ↦ indirection_closure(a_pap)]
```

22 State Transition: ReturnInt with Update Frame on top of the stack

[todo] we should really disallow / optimized this scenario as much as possible (repeated boxing and unboxing of primitves)

```
ReturnInt k, (uf:s), h, σ

⟹

ReturnInt k, s, h'', σ

where:

  a_thunk = uf.thunk

  h' = h[a_boxed ↦ BoxedInt# k]

  h'' = h'[a_thunk ↦ indirection(a_boxed)]
```

23 GC Codes

if assuming 2-space copying algorithm: each closure needs evacuation and scavenge codes

23.1 evacuation code

copy closure to to-space, overwrite from-space copy with a forwarding pointer, and returns the new address in to-space

forwarding pointer implementable with a closure (with destination pointer as the closure payload) with its evacuation code that returns the to-space destination address $\implies$ using uniform representation of closure, all closures need to have space to store atleast 1 word in payload

23.2 scavenge code

once a closure's evacuation code is complete, run scavenge code on the closure in the to-space

purpose: recursively run evacuation code on all items in its payload-that are pointers which also returns the new address of the evacuated object in to-space, then update the pointers in the payload with the updated addresses in to-space

note: these compiled codes can be made layout-aware of its payload and not require extra runtime assistance/interpretation

23.3 indirection closure

its evacuation code jumps to the destination closure's evacuation code (if the destination is an indirection closure, then the indirection closure's evacuation code will have an extra indirection that returns its target closure's address in to-space $\implies$ removal of the indirection after GC cycle completes)

References

[1] Simon L. Peyton Jones. Implementing lazy functional languages on stock hardware: The Spineless Tagless G-machine. *Journal of Functional Programming*, 2(2):127–202, 1992.

[2] Simon Marlow and Simon Peyton Jones. Making a fast curry: push/enter vs. eval/apply for higher-order languages. *Journal of Functional Programming*, 16(4-5):415–449, 2006.